

Chapter 7* Ray Tracing Practice(1)

• 核心

• 光线追踪渲染流水线

Basic Ray Tracing

- **步骤一：**加载模型。模型的相关属性包括模型的材质，对于透射材质需要知道其折射率，对于非镜面材质需要知道其diffuseColor
- **步骤二：**开始主渲染流水线
 - **步骤2.1：**计算从每一个像素投射出去的光线方向，需要用到的参数有fov（纵向视场角），AspectRatio（视口宽高比）
 - **步骤2.2：**向计算所得方向投射出光线并计算当前像素点的着色情况，将计算所的颜色存储在帧缓冲区中
 - **步骤2.2.1：**枚举空间中的每一个物体，找到与当前光线产生第一个交点的物体（对于不同的物体有不同的求交方法，如球体、三角形面）。若不存在这样的物体则返回环境颜色，追踪结束；反之进入步骤2.2.2
 - 备注：三角形面上任意点表示方法为 $p+au+bv$ ($0 < a, b, a+b < 1$)，其中 u, v 为边向量， p 为 u, v 共用的点，不要把点忘了
 - **步骤2.2.2：**提取hit point对应object的材质。若为镜面材质则计算反射光线并继续光线追踪；若为透射材质则计算折射光线并继续光线追踪；若为漫反射材质则停止追踪并返回计算得到的hit point着色情况
 - **步骤2.2.3：**继续光线追踪时，若迭代次数超过某一值，则返回无色 `Vector3f(0, 0, 0)`，反之返回步骤2.2.1继续光线追踪
 - **步骤2.3：**输出帧缓冲区中的内容

• 加速结构

Accelerating Structure, target at the calculation of the intersections of the ray and objects(step 2.2.1)

• BVH加速结构

Bounding Volume Hierarchy

- **步骤一：**加速结构的构建。递归构建，每个节点记录左儿子节点的指针、右儿子节点的指针、当前节点的AABB，对于叶子节点，额外记录该节点所包含的所有Objects
 - 对于每层的划分可以采用中点划分，等量划分或SAH划分
- **步骤二：**加速结构的使用。利用加速结构求解与当前光线第一个相交的物体及相关信息
 - **步骤2.1：**从整个加速结构（一般为树形结构）的根节点开始遍历
 - **步骤2.2：**对于当前遍历到的节点node，若其不为叶子节点，则分别检查其左右儿子的AABB与当前光线的交情况，若有交则继续遍历直至其为叶子节点

备注：检查AABB与光线交时需要注意光线direction每一维的正负，这会影响到光线从pMin进入AABB还是从pMax进入AABB

- **步骤2.3：**对于当前遍历到的节点node，若其为叶子节点，则计算当前光线与其中所有物体的交，返回距离光线出发点最近的交点信息。交点信息包括交点坐标、交点处法向量、距离（光线 $r=o+td$ 中的 t ）、交点所在物体、交点处材质、是否有交（记录是否有交的原因是无交点时也会返回交点信息，此时记录是否有交的bool变量值为false）
- **步骤2.4：**回溯时，对于每个非叶子节点node，若两个儿子都返回了交点，则同步骤2.3中一样保留并返回距离光线出发点最近的交点信息

• SAH划分优化BVH加速结构

Surface Area Heuristic

- **概念引入：**引入估价函数 $C(b), 0 \leq b < B$ ，其含义为用（超）平面将当前节点对应的空间沿其跨度最长的坐标轴划分为 B 个buckets，取其中划分了第 b 与第 $b + 1$ 个bucket的（超）平面作为最终选定的划分方案所需期望代价。另外，认为 $C(0)$ 为不对该节点进行划分，直接计算当前光线与全部物体的交所需的代价
- **理论推导**
 - **形式一：** $C_{node}(0) = \#Objects$, $C_{node}(b) = p(lnode) \times \min\{C_{lnode}\} + p(rnode) \times \min\{C_{rnode}\} + C_{ray-AABB-intersection}, 0 < b < B$
 - **形式二：**由于在SAH优化下每一层划分的坐标轴不同，因此完整计算 $\min\{C_{lnode}\}$ 项与 $\min\{C_{rnode}\}$ 项不够现实，于是采用简化近似：假定对于以某一点为根的子树，它最小期望代价与其内部的物体数成正比，而这里只是计算相对代价，故比例系数可以忽略。简化后的代价计算式可写作 $C_{node}(b) = p(lnode) \times \#Objects_{lnode} + p(rnode) \times \#Objects_{rnode} + C_{ray-AABB-intersection}$
 - **形式三：**SAH认为选中某个子节点的概率为该子节点AABB的表面积与当前节点AABB的表面积之比，因此形式二中的公式中的概率 p 可以被代换，从而得到新的公式： $C_{node}(b) = \frac{S(lnode)}{S(node)} \times \#Objects_{lnode} + \frac{S(rnode)}{S(node)} \times \#Objects_{rnode} + C_{ray-AABB-intersection}$ 。注意式中最后的常数项也可删除，因为相对大小比较不需要知道其严格大小
- **实现流程**
 - **步骤一：**在建立节点时，若当前节点包括的物体数大于2，则找到当前节点AABB跨度最大的坐标轴
 - **步骤二：**对所有物体进行排序（依据每个物体AABB的中心在该坐标轴上的大小关系）
 - **步骤三：**计算每个bucket中的物体（依据每个物体的AABB的中心与划分（超）平面在该坐标轴上的大小关系）
 - **步骤四：**计算buckets的前缀与后缀AABB
 - **步骤五：**对于每个划分（超）平面，依据该点的前缀buckets与后缀buckets以及理论推导形式三中的公式计算得到代价。在此过程中记录下最小划分代价的

(超) 平面

- 步骤六：依据最小代价（超）平面划分物体集，继续递归建立节点

- 问题

- 问题一：程序出现段错误。经调试发现部分物体的AABB中心相距很近，极端情况下聚集在边界的bucket上，导致无法继续划分（可能多个物体存在于一个叶子节点中），从而递归溢出
- 解决一：用每个物体的AABB中心做包围盒，以包围盒最长边为该次选择的划分坐标轴。这种方法可以正确选择跨度最长的坐标轴。同时选择更多的划分buckets数（1000个）
- 问题二：解决了递归溢出问题后，程序仍然出现段错误。经调试发现排序后最后一个物体会加入到最后一个bucket之后的一个bucket，导致空间越界访问
- 解决二：注意浮点数比较带来的误差！引入epsilon即可

- 最终效果

- BVH: 7.2s
- BVH+SAH: 6.3s
- 优化幅度：12.5%

- 双向反射分布函数

Bidirectional Reflectance Distribution Function

- 情境引入：对于一个面光源以及一个半球，想要知道该光源对该半球的辐射通量

- 问题拆解

- 拆解1：辐射通量的定义是 $\Phi = \frac{dQ}{dt}$ ，直接求解过于复杂，因此引入新定义的变量：辐照度 E ，它的定义是 $E = \frac{d\Phi}{dA}$ ，即每单位面积的辐射通量。于是若知道半球面上每一单位面积的辐射通量 E ，就可以通过 $\int_{\Omega^+} E dA$ 求解出整个半球的辐射通量（ Ω^+ 是半球面）。
- 情境转换：实际上在实时渲染中计算辐射通量 Φ 的必要性远低于辐照度 E ，因为我们并不需要知道单位时间里有多少能量被辐射到整个物体上，而更需要知道物体上每一单位面积接收到多少能量，从而得到它沿某个方向辐射出的能量，并据此计算每一个像素的着色
- 拆解2：现在的第一个问题是求解半球上每一单位面积的辐照度 E 。由于光源是面光源，直接求解还是过于复杂，因此再次引入新定义的变量：辐亮度 L ，它的定义是 $L = \frac{dE}{dw \cos \theta}$ ，也可以进一步写作 $L = \frac{d^2\Phi}{dA dw \cos \theta}$ ，即每单位面积每单位立体角的辐射通量（注：此处每单位立体角是以半球上某一点（或单位面积）为顶点，以面光源为底构成的锥体的立体角； $\cos \theta$ 表示投影，用于计算非垂直照射下的能量损失）。于是对于半球上每一单位面积，它的辐照度是 $\int_{H^2} L dw_i \cos \theta_i$ ，其中 H^2 表示光源平面， w_i 表示光源上某一点到半球上这一点的立体角， θ_i 表示投影角
- 拆解3：现在的第二个问题是在得到半球面上某一单位面积的辐照度之后，如何计算该单位面积沿某个方向辐射出的能量。这里的第一个假设是：对于不同从不同方向入射到同一单位面积的辐射，即使它们在该点上产生了相同的辐照度，它们产生的辐亮

度在各方向上的分布也是不同的。基于这个假设，我们不能直接根据该单位面积总的辐照度计算它辐亮度的分布，因此我们需要将该点的辐亮度分解为由不同光源微元在该点产生的辐照度所导致的辐亮度的叠加。比如对于面积为 dA 的面光源，他在球面某一单位面积上产生的辐照度 $dE = Ldw_i \cos \theta_i$ ，而该辐照度所导致的辐亮度为 dL_o ，将面光源的所有面积微元在该处所导致的辐亮度叠加就可以得到该点的总辐亮度 L_o 。这里的**第二个假设**是：面积为 dA 的面光源在该处产生的辐照度 dE 与 dA 成正比，而该比例系数受到入射方向、观察方向以及入射点三个因素的影响（表面均匀的材质只受入射方向以及观察方向的影响），因此将该比例系数定义为 $f(p, w_i \rightarrow w_r)$ 。于是有 $f(p, w_i \rightarrow w_r) = \frac{dL_o}{dE} = \frac{dL_o}{Ldw_i \cos \theta_i}$ （注：该计算式并不是 f 的定义式， f 的值并不受式中变量的影响，它是**材质的固有属性**）。根据该比例系数，我们可以计算该单位面积沿某个方向的总辐亮度 $L_o = \int_{H^2} f(p, w_i \rightarrow w_r) dE = \int_{H^2} L_i(q, w_i) f(p, w_i \rightarrow w_r) dw_i \cos \theta_i$ 。式中的 f 函数就是双向反射分布函数，简称BRDF

• 性质

- **交换律**： $f(p, w_i \rightarrow w_r) = f(p, w_r \rightarrow w_i)$
- **能量守恒**： $\int_{\Omega^+} f(p, w_i \rightarrow w_r) \cos \theta_i dw_r \leq 1$ ，该式是 $\int_{\Omega^+} L_i(q, w_i) f(p, w_i \rightarrow w_r) dw_r \cos \theta_i \leq L_i(q, w_i)$ 的简化。注意，这里与上方的积分变量不同：上方的积分变量是入射光角度 w_i ，最终结果是所有方向的光源微元在该点产生的辐照度所导致的在 w_r 方向上的辐亮度；此处的积分变量是出射光角度 w_r ，最终结果是从某一方向打来的辐亮度垂直于该单位平面的分量所产生的辐照度所导致的辐亮度在半球面中各个方向的分量之和，显然这个结果应当小于入射的辐照度垂直于该单位平面的分量

• 路径追踪渲染流水线

Path Tracing

• 渲染流水线

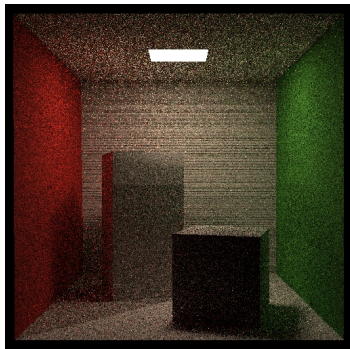
- **步骤一**：加载模型。模型的相关属性包括模型的材质，对于透射材质需要知道其折射率，对于非镜面材质需要知道其diffuseColor
- **步骤二**：开始主渲染流水线
 - **步骤2.1**：计算从每一个像素投射出去的光线方向，需要用到的参数有fov, AspectRatio
 - **步骤2.2**：向计算所得方向投射出光线并计算当前像素点的着色情况，将计算所的颜色存储在帧缓冲区中
 - **步骤2.2.1**：枚举空间中的每一个物体，计算光线与它们的交点，并选择最近的物体。当然也可以选用BVH, SAH加速结构加速这个过程
 - **步骤2.2.2**：若与光源相交，则直接返回光源颜色，否则进入下一步
 - **步骤2.2.3**：（这里只暂且考虑漫反射材质）有两种光来源，一种是光源，另一种是其它物体的漫反射。对于光源，直接采样光源上任意一点，然后检测当前点与光源上采样点之间是否有障碍，若无则直接返回用蒙特卡洛积分算出的Rendering Equation结果；对于其他物体，继续向他们投射光线，将它们

返回的结果作为光源发出的颜色，同样用蒙特卡洛积分算出Rendering Equation结果并返回，这里还需额外加入俄罗斯轮盘的思想

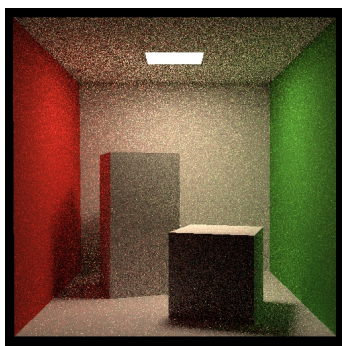
- 步骤2.3：输出帧缓冲区中的内容

• 问题记录

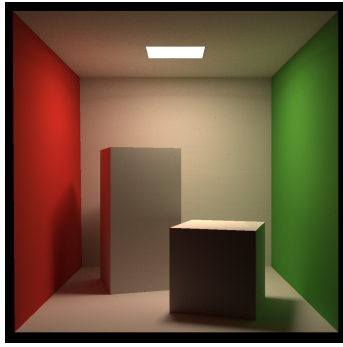
- 问题一：部分平面未在渲染图像中显示
- 解决一：判断光线与AABB相交时将 $t_{enter} \leq t_{exit}$ 纳入合法情况，因为存在部分与坐标平面平行的平面，它们的包围盒的pMin与pMax在某一维上相同
- 问题二：光源在渲染图像中不显示（全黑）
- 解决二：当castRay中投射出的光线第一次就击中光源时，直接返回光源的颜色
- 问题三：画面中出现黑色横向条纹



- 解决三：Assignment PDF文档中提示可能是判断光线是否被挡的边界情况出现问题。注意到当判断当前点与光源上某一点的连线是否被其他障碍物挡住时，我将连线的出发点向着光源上一点移动了EPSILON距离，而之后判断交点到当前点距离与当前点到光源上一点的距离时我将后者减去了EPSILON，这实际上与之前的EPSILON抵消了，但之前的EPSILON也是必要的，因此这里需要改为减去 $2.0 * EPSILON$ 。改完之后发现还不行，直接将EPSILON从 $1e-5$ 调至 $1e-4$ ，黑色条纹消失
- 问题四：噪点过多



- 解决四：提升spp
- 问题五：仍然存在隐约的黑色横纹，且物体边缘存在大量锯齿



- **解决五：**再次调高EPSILON，且将SPP次CastRay分为若干个方向投射出去（MSAA），继续提高SPP

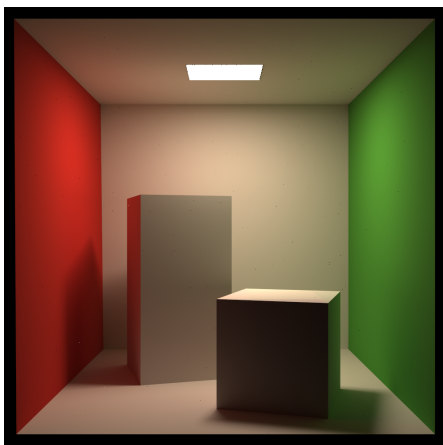
• 多线程优化

- **适用场景：**程序需要进行多次计算，且每次计算的结果对其它计算过程不产生影响或影响很小。如在渲染时需要计算每一个像素的着色，每次计算过程相似且独立，因此可以使用多线程优化实现并行计算。
- **多线程问题：**在本次作业中，渲染时除了计算每个像素的着色还需要实时显示渲染进度。而渲染进度是所有线程共用的变量，因此在更新渲染进度时必须进行上锁与解锁，以防进程冲突导致对渲染进度的读脏情况

• 运行结果

- **无优化+SAH：** 512x512, 8spp, 4核虚拟机, 耗时795s
- **多线程优化+SAH：** 512x512, 8spp, 4核虚拟机, thread/mutex, 耗时681s
- **多线程优化+SAH：** 512x512, 8spp, 4核虚拟机, OpenMP+O3, 耗时276s
- **在优化三的基础上仿照优秀作业链接中在 `get_random_float` 函数中的变量前以 `static` 修饰：** 512x512, 8spp, 4核虚拟机, OpenMP+O3, 耗时8s
- **综合OpenMP+O3+SAH+Static：** 1024x1024, 1024spp, 4核虚拟机, 耗时2761s
- **最终效果图运行耗时：** 1024x1024, 16384SPP, 4核虚拟机, 耗时42369s

• 最终效果图



1024x1024, 16384SPP

- **优秀作业链接：** [【GAMES101】作业7（提高）路径追踪 多线程、Microfacet（全镜面反射）、抗锯齿 ycr的帐号的博客-CSDN博客](#)

- 其它

- 段错误

- 情况一：数组访问越界
 - 情况二：使用为空的指针
 - 情况三：递归层数过多导致栈溢出

- 缩短渲染时长

- 程序性能优化

- 算法优化
 - 变量定义优化（多次使用的函数中定义新变量，static?）
 - 多线程优化
 - 硬件契合性优化

- 硬件性能优化

- 使用云端服务器